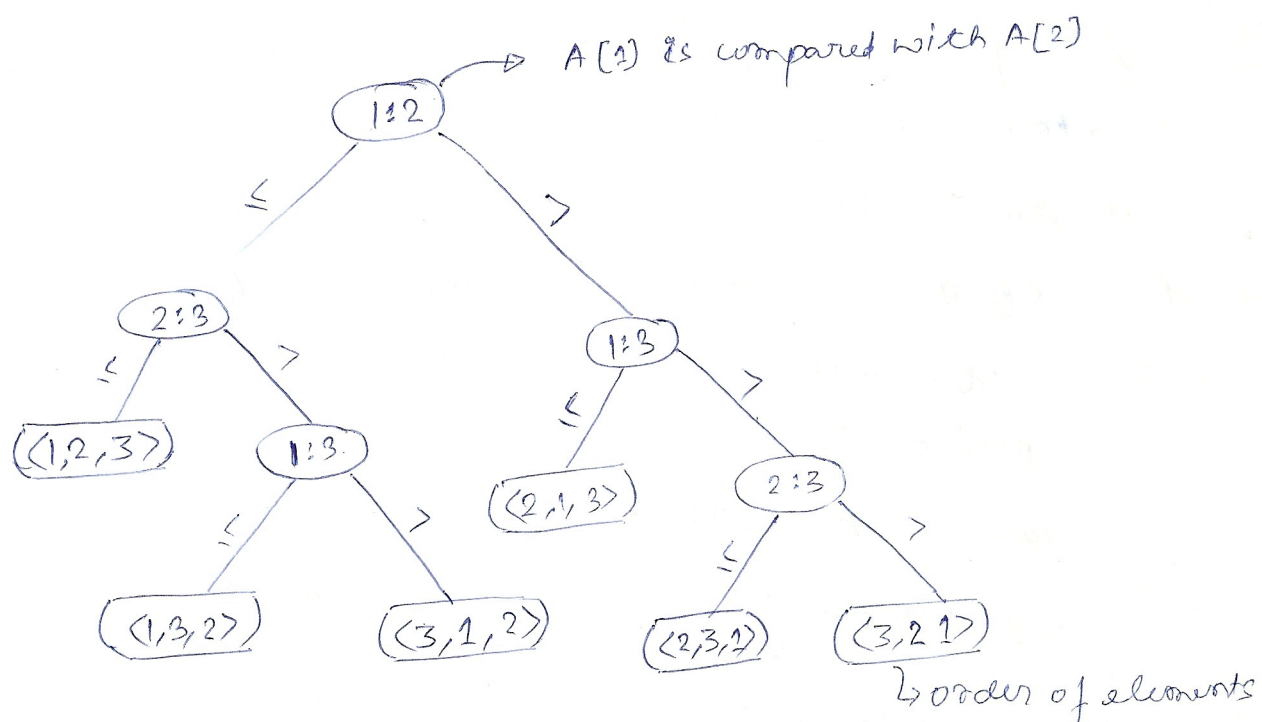


① $\rightarrow$ A sorting algorithm that determines the sorted order based on comparisons between the I/P elements, is called a comparison sort. eg: insertion sort, merge sort, heapsort.

$\rightarrow$ The decision-tree model

Comparison sorts can be viewed in terms of decision trees. A decision tree is a full binary tree that represents the comparisons between elements performed by a sorting algorithm.



$\rightarrow$ Decision tree for insertion sort operating on 3 elements.

A leaf annotated by the permutation $\langle \pi(1), \pi(2), \dots, \pi(n) \rangle$ indicates the sorted order $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$

$\rightarrow$ The execution of a sorting algorithm corresponds to tracing a path from the root of the decision tree to a leaf (which represents that permutation of the I/P no's that is the sorted order).

$\rightarrow$ A comparison sort operating on n elements must be capable of producing $n!$ permutations (because each could be the correct sorted order depending upon the n elements). Thus, a decision tree should have $n!$ leaves and each should be reachable from the root.

→ The length of the longest path from the root of a decision tree to its leaves represents the worst-case no. of comparisons that must be performed. (2)

→ Any comparison sort algorithm requires $\Omega(n \lg n)$ comparisons in the worst case.

no. of required permutations $= n! \leq 2^h = \text{max. no. of leaves in a binary tree of height } h$

$$h \geq \lg n!$$

$$= \Omega(n \lg n)$$

② Counting sort

→ Counting sort assumes that each of the n I/P elements is an integer in the range 0 to K .

→ First of all, it determines the no. of elements less than x .
For eg., if there are ¹⁷~~18~~ elements less than x , then x belongs in O/P position 18.

→ I/P array is $A[1..n]$, the array $B[1..n]$ holds the O/P and $C[0..K]$ provides the storage for counts.

COUNTING-SORT(A, B, K)

```
1 for  $i \leftarrow 0$  to  $K$ 
2   do  $c[i] \leftarrow 0$ 
```

▷ $c[i]$ is supposed to hold the count of the no. of occurrences of ~~$A[i]$~~ element i in array A .

```
3 for  $j \leftarrow 1$  to  $\text{length}[A]$ 
4   do  $c[A[j]] \leftarrow c[A[j]] + 1$ 
```

```
5 for  $i \leftarrow 1$  to  $K$ 
6   do  $c[i] \leftarrow c[i-1] + c[i]$ 
```

▷ now $c[i]$ contains the no. of elements less than or equal to i .


```
7 for j ← length[A] downto 1
8   do B[c[A[j]]] ← A[j]
9     c[A[j]] ← c[A[j]] - 1
```

→ For any $A[j]$, $c[A[j]]$ gives the no. of elements of A that are less than or equal to $A[j]$. Hence, the correct position of $A[j]$ in the o/p array B is $c[A[j]]$. Because, there could be multiple elements in A equal to $A[j]$, and because they have already been considered in $c[A[j]]$ we decrement $c[A[j]]$ after putting $A[j]$ into B . The next element with value equal to $A[j]$, if one exists, would go to the position immediately before $A[j]$ in the array B .

→ The running time of counting sort is $\Theta(n+k)$. We use counting sort when $k = O(n)$ so that the running time is $\Theta(n)$.

→ Counting sort beats the $\Omega(n \lg n)$ runtime of comparison sorts.

→ Counting sort is "stable": i.e. numbers with the same value appear in the o/p array in the same order as they do in the i/p array. This could be important when numbers have satellite data associated with them. Counting sort is used as a subroutine in radix sort and its stability is crucial to radix sort's correctness.

③ Radix sort

→ i/p: an array of n numbers of d digits each.

→ Radix sort, sorts the n numbers by the least significant digit first and ends by sorting the n numbers by the most significant digit. d -passes through the n numbers are required where every pass uses some sorting algorithm like counting sort. Counting sort is used, because it is stable, and helps radix sort in making it correct.

④

→ The following procedure assumes that digit 1 is the least significant digit and digit d is the most significant one.

RADIX-SORT (A, d)

1 for $i \leftarrow 1$ to d

2 use a stable sort to sort array A on digit i

→ If each digit is in the range 0 to $K-1$ (total K values) each call to counting sort will take $\Theta(n+K)$ time. Since the total such calls ~~is~~ ^{is} d , the runtime of radix sort is $\Theta(d(n+K))$

→ If each of the n numbers is b -bits and we decide to group r bits into a digit, no. of digits $d = b/r$. The running time of radix sort then would be $= \Theta(b/r(n+2^r))$

④ Bucket sort

Bucket sort is used in case the VP is distributed uniformly over the interval $[0, 1)$. Bucket sort divides the interval $[0, 1)$ into n buckets, and then distributes the n VP numbers into the buckets. Because the numbers are uniformly distributed over $[0, 1)$, the no. of elements that fall into a bucket is low. To produce the o/p, we simply sort the numbers in every bucket and go through the buckets in order, listing the elements in each.

→ VP is a n element array A such that $0 \leq A[i] < 1$.

An auxiliary array $B[0..n-1]$ of linked lists (buckets) is used.

BUCKET-SORT (A)

1 $n \leftarrow \text{length}[A]$

2 for $i \leftarrow 1$ to n

3 do insert $A[i]$ into list $B[\lfloor n A[i] \rfloor]$

4 for $i \leftarrow 0$ to $n-1$

5 do sort list $B[i]$ with insertion sort

6 concatenate the lists $B[0], B[1], \dots, B[a-1]$ together ⑤
in order

→ The expected time for bucket sort is $\Theta(n)$.

